

Array Equalization Study Guide

MINIMUM OPERATIONS & VALUE INSERTION STRATEGY

Perfect! Now that you've understood the idea, here's a complete explanation in the proper order that you can use for your notes or in an interview.

Problem Intuition

We are given an array of distinct integers. We have to:

- Insert one new distinct integer into the array.
- Choose one positive integer x .
- In one operation, add x to exactly one element.
- Make all $n+1$ elements equal using the minimum number of operations.

Step 1: Understand what the operation means

Suppose $x = 4$.

If an element is `5`, it can become: `5 → 9 → 13 → 17 → ...`

Notice something? It can only reach numbers that differ by multiples of 4.

So if the final target is T , then $(T - a_i)$ must be divisible by x . Otherwise, that element can never reach T .

This is our first important observation: Every $(Target - a_i)$ must be divisible by x .

Step 2: Which target should we choose?

Since we can only add, we can never decrease any element. Therefore:

$$Target \geq \text{maximum element}$$

For example, in the array `[2, 8, 14]`, we can never make everything equal to `10` because `14` cannot decrease. So the target is always at least the current maximum.

Step 3: Which x should we choose?

Suppose $Target = 14$. The differences are: `12`, `6`, and `0`.

If $x = 2$, the operations required are $12/2 = 6$ and $6/2 = 3$. The total is large.

If $x = 6$, the operations required are $12/6 = 2$ and $6/6 = 1$. This is much smaller.

So we want the largest possible x . But x must divide every difference. The largest number dividing all differences is exactly:

$$\text{GCD}(\text{differences})$$

Hence: $x = \text{gcd}(\max - a_i)$. This is the biggest valid x , giving the minimum number of operations.

Step 4: Why is the target taken as the maximum?

Suppose the maximum is M . Can making the target larger help?

Suppose we choose $\text{Target} = M + kx$. Every original element now needs:

$$(M - a_i + kx) / x = (M - a_i) / x + k \text{ operations.}$$

So every original element requires k extra operations. The new inserted element needs 0 operations. Therefore, the total operations increase by $n \times k$ or remain the same in special cases. Hence, making the target larger never gives a better answer. Therefore, we can safely compute everything assuming:

$$\text{Target} = \text{current maximum}$$

Step 5: Computing operations

Once we know $\text{Target} = mx$ and $x = \text{gcd}(\text{all differences})$, each element needs $(mx - a_i) / x$ operations. So:

```
ans += (mx - a[i]) / x;
```

Step 6: Now comes the inserted element

We must insert one more number. Suppose $\text{Target} = 14$ and $x = 6$. Which numbers can eventually become 14?

Only `14, 8, 2, -4, -10, ...` because every one differs from 14 by a multiple of 6.

So every valid inserted number is of the form: $mx - k * x$

Step 7: Which inserted number is best?

Operations for the new number are $(mx - new) / x$. To minimize operations, we want the inserted number as close to mx as possible.

So we try $mx - x$. If it is already present, we try $mx - 2x$. If that is also already present, we try $mx - 3x$, until we find a missing number. This is exactly what the code does:

```
long long cur = mx - x;
while(st.count(cur))
    cur -= x;
```

Finally, we add its operations to the answer:

```
ans += (mx - cur) / x;
```

DOUBT 1: WHY DON'T WE INSERT A NUMBER GREATER THAN THE MAXIMUM?

Example: Array `[2, 8, 14]`

If we insert `20`, the target becomes `20`.

Operations: $2 \rightarrow 20$ (=3), $8 \rightarrow 20$ (=2), $14 \rightarrow 20$ (=1), $20 \rightarrow 20$ (=0). **Total = 6**

Instead, if we insert `-4`, the target remains `14`.

Operations: $2 \rightarrow 14$ (=2), $8 \rightarrow 14$ (=1), $14 \rightarrow 14$ (=0), $-4 \rightarrow 14$ (=3). **Total = 6**

The answer is the same. If we insert even larger numbers, operations only increase. So searching below the maximum is enough.

DOUBT 2: IN THE CODEFORCES EXAMPLE, ARRAY [1,2,3], THE OFFICIAL SOLUTION INSERTS 4. WHY?

Because after insertion, the array becomes `[1, 2, 3, 4]` and the maximum becomes `4`. Making everything equal to 4 is valid. Operations: $3 + 2 + 1 + 0 = 6$.

Our algorithm inserts `0`. The operations become: $2 + 1 + 0 + 3 = 6$. Both yield the exact same answer and both are optimal. The official statement is only showing one optimal solution; our implementation finds another valid one.

DOUBT 3: WHY DO WE ONLY SEARCH DOWNWARD?

Because every valid inserted number is $mx - k * x$. The first missing one gives the fewest operations. Searching downward is easy and guarantees the minimum. Searching upward is unnecessary because it never gives a better answer.

Complete Algorithm

Step 1

Read all numbers.

Step 2

Find $mx = \text{maximum element}$.

Step 3

Compute $x = \gcd(mx - a[i])$.

Step 4

Count operations for every existing element:

```
ans += (mx - a[i]) / x;
```

Step 5

Store every number in a set.

Step 6

Start from $cur = mx - x$. If it already exists, perform $cur -= x$. Repeat until a missing value is found.

Step 7

Add operations for the inserted element:

```
ans += (mx - cur) / x;
```

Step 8

Print the final answer.

FINAL INTERVIEW INTUITION (30 SECONDS)

Since we can only increase values, every element must reach a common target greater than or equal to the maximum. For an element to reach the target by repeatedly adding x , the difference $(\text{target} - a_i)$ must be divisible by x . To minimize operations, we choose the largest possible valid x , which is the GCD of all differences from the maximum. The operations for existing elements are then fixed. Finally, we insert the closest missing number below the maximum that also differs from it by a multiple of x , because that minimizes the additional operations required for the new element. This yields the minimum total number of operations.